Национальный исследовательский университет
«Высшая школа экономики»

EcoLab1

Создание компонентов, реализующих решение систем обыкновенных дифференциальных уравнений методом Рунге-Кутты 4-го порядка

Задание 36

Выполнил:
Боронин Фёдор Дмитриевич
группа 24ТИДИ

Преподаватель:
Башев Владимир Николаевич

2026

Содержание

1. Постановка задачи
2. Алгоритмы
3. Интерфейс и компоненты
4. Клиент
5. Проверка точности
6. Производительность
7. Сборка и запуск
8. Выводы
9. Источники

1. Постановка задачи

Решение систем обыкновенных дифференциальных уравнений (ОДУ) методом Рунге-Кутты 4-го порядка, аналог функции ode45 в MATLAB.

Требования, которые следуют из общей постановки:

- компонент пишется на C89 по технологии АОМК (ACOM) поверх Eco.Framework;
- внутри компонента нельзя использовать стандартную библиотеку (libc), в том числе `malloc` и `math.h`;
- нужно самому спроектировать интерфейс и типы данных;
- клиент - консольное приложение (unit-тест или калькулятор);
- в отчете нужны таблицы сравнения с `math.h` или MATLAB и замеры производительности;
- если у задачи есть несколько алгоритмов, каждый оформляется отдельным компонентом, а клиент выбирает нужный.

В работе сделано два компонента с одним и тем же интерфейсом. Первый, `Eco.Lab1`, реализует классический метод Рунге-Кутты 4-го порядка с постоянным шагом. Второй, `Eco.Lab1DP`, реализует метод Дормана-Принса 4(5) с автоматическим выбором шага. Второй компонент добавлен потому, что ode45 в MATLAB построен именно на методе Дормана-Принса, а классический метод с постоянным шагом является его аналогом только отчасти.

2. Алгоритмы

2.1. Задача

Решается задача Коши для системы из n уравнений первого порядка:

$$y' = f(t, y),\ \ y(t_0) = y_0,\ \ y \in R^n$$

Уравнение более высокого порядка сводится к такой системе заменой переменных. Например, $x'' = -x$ превращается в систему $y_1' = y_2$, $y_2' = -y_1$.

2.2. Метод Рунге-Кутты 4-го порядка

Один шаг длины h от точки (t_n, y_n) считается по формулам:

$$k_1 = f(t_n\ , y_n\ )$$
$$k_2 = f(t_n\ + h/2, y_n\ + h/2 \cdot k_1)$$
$$k_3 = f(t_n\ + h/2, y_n\ + h/2 \cdot k_2)$$

$$k_4 = f(t_n + h, y_n + h \cdot k_3)$$

$$y_{n+1} = y_n + h/6 \cdot (k_1 + 2k_2 + 2k_3 + k_4)$$

На шаг уходит 4 вычисления правой части. Ошибка на одном шаге имеет порядок h^5, на всем отрезке h^4. Значит, если уменьшить шаг в 2 раза, ошибка должна упасть примерно в 16 раз. Это свойство проверяется в тестах (раздел 5.1).

Недостаток метода в том, что шаг постоянный и задается заранее. Метод не знает, какую ошибку он допускает, поэтому шаг приходится подбирать по самому трудному участку решения.

2.3. Метод Дормана-Принса

Это тоже метод Рунге-Кутты, но на каждом шаге он считает сразу два решения: 5-го и 4-го порядка. Оба получаются из одних и тех же семи значений $k_1 \ldots k_7$, отличаются только веса в итоговой сумме. Разность двух решений используется как оценка ошибки шага.

Стадии считаются по общей формуле:

$$k_i = f(t_n + c_i h, y_n + h \cdot \Sigma\, a_{ij}\ k_j\ ),\ \ j = 1 \ldots i-1$$

Коэффициенты (таблица Бутчера) приведены в таблице 1.

Таблица 1. Коэффициенты метода Дормана-Принса

c_i	a_{i1}	a_{i2}	a_{i3}	a_{i4}	a_{i5}	a_{i6}
0						
1/5	1/5					
3/10	3/40	9/40				
4/5	44/45	−56/15	32/9			
8/9	19372/6561	−25360/2187	64448/6561	−212/729		
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656	
1	35/384	0	500/1113	125/192	−2187/6784	11/84

Решение 5-го порядка берется с весами из последней строки таблицы:

$$y_{n+1} = y_n + h \cdot (35/384 \cdot k_1 + 500/1113 \cdot k_3 + 125/192 \cdot k_4 - 2187/6784 \cdot k_5 + 11/84 \cdot k_6)$$

Оценка ошибки равна разности решений 5-го и 4-го порядка:

$$e = h \cdot (71/57600 \cdot k_1 - 71/16695 \cdot k_3 + 71/1920 \cdot k_4 - 17253/339200 \cdot k_5 + 22/525 \cdot k_6 - 1/40 \cdot k_7)$$

Последняя стадия k_7 считается в точке $(t_n + h, y_{n+1})$, то есть она совпадает с k_1 следующего шага. Поэтому после принятого шага k_7 переносится в k_1 и заново не

вычисляется. В итоге на каждую попытку шага приходится 6 вызовов правой части, плюс один вызов в самом начале.

2.4. Выбор шага

Пользователь задает не шаг, а точность: относительный допуск RelTol и абсолютный AbsTol. Для каждой компоненты решения считается масштаб, и ошибка нормируется на него:

$$sc_i = AbsTol + RelTol \cdot \max(|y_{n,i}|, |y_{n+1,i}|)$$

$$err = \max |e_i| / sc_i$$

Если err ≤ 1, шаг принимается, иначе отклоняется и повторяется с меньшим h. В обоих случаях новый шаг вычисляется так:

$$h_new = h \cdot \min(5, \max(0.2, 0.9 \cdot err\text{^}(-1/5)))$$

Множитель 0.9 дает запас, а границы 0.2 и 5 не позволяют шагу меняться слишком резко. Сразу после отклоненного шага увеличивать h запрещено. Последний шаг укорачивается так, чтобы попасть ровно в конец отрезка. Если параметры не заданы, используются RelTol = 10^{-3} и AbsTol = 10^{-6}, как в ode45.

Для этой формулы нужен корень 5-й степени, а функцию `pow` из `math.h` использовать нельзя. Поэтому корень считается методом Ньютона. Сначала аргумент приводится к отрезку [1, 32) умножением или делением на 32 (при этом сам корень меняется ровно в 2 раза), затем делается 8 итераций:

$$r \leftarrow (4r + x / r^4) / 5$$

3. Интерфейс и компоненты

3.1. Интерфейс IEcoAdvancedMath

Интерфейс описан в файле `SharedFiles/IEcoAdvancedMath.h`. Он наследует `IEcoUnknown` (первые три метода) и добавляет шесть своих: по одному шагу и по одному решению на отрезке для каждого из трех типов чисел.

```
typedef int16_t (ECOCALLMETHOD *EcoOdeFunc)(/* in */ voidptr_t ctx, /* in */ double_t
t,
        /* in */ const double_t* y, /* out */ double_t* dydt, /* in */ uint32_t n);

typedef struct EcoOdeOptions {
    double_t Step;
    double_t RelTol;
    double_t AbsTol;
    uint32_t MaxSteps;
} EcoOdeOptions;

typedef struct EcoOdeStats {
    uint32_t Points;
    uint32_t Steps;
    uint32_t Rejected;
    uint32_t Evaluations;
} EcoOdeStats;

typedef struct IEcoAdvancedMathVTbl {
    /* IEcoUnknown */
    int16_t (ECOCALLMETHOD *QueryInterface)(IEcoAdvancedMathPtr_t me, const UGUID*
riid, voidptr_t* ppv);
    uint32_t (ECOCALLMETHOD *AddRef)(IEcoAdvancedMathPtr_t me);
    uint32_t (ECOCALLMETHOD *Release)(IEcoAdvancedMathPtr_t me);

    /* IEcoAdvancedMath */
    int16_t (ECOCALLMETHOD *Step)(me, f, ctx, n, t, h, y, yOut);
    int16_t (ECOCALLMETHOD *Stepf)(...);
    int16_t (ECOCALLMETHOD *Stepl)(...);
    int16_t (ECOCALLMETHOD *Solve)(me, f, ctx, n, t0, t1, y0, opt, capacity, tOut,
yOut, stats);
    int16_t (ECOCALLMETHOD *Solvef)(...);
    int16_t (ECOCALLMETHOD *Solvel)(...);
} IEcoAdvancedMathVTbl;
```

В листинге сигнатуры методов сокращены, полный текст находится в исходниках. Пояснения к параметрам:

- `f` - правая часть системы. Это функция клиента, компонент вызывает ее по указателю. Если она возвращает не ноль, решение останавливается, и этот код возвращается клиенту.
- `ctx` - произвольный указатель, который компонент передает в `f` без изменений. Через него клиент передает параметры задачи.
- `n` - число уравнений в системе.
- `opt` - параметры: шаг, допуски, ограничение числа шагов. Можно передать 0, тогда берутся значения по умолчанию.
- `capacity`, `tOut`, `yOut` - массивы под результат выделяет клиент, `capacity` задает, сколько точек в них помещается. В `tOut` записываются моменты времени, в `yOut` подряд значения решения, по n чисел на точку.
- `stats` - статистика: число точек, шагов, отклоненных шагов и вызовов правой части.

Массивы под результат выделяет клиент по двум причинам. Во-первых, внутри компонента нельзя вызывать `malloc`. Во-вторых, число точек у адаптивного метода заранее неизвестно, и клиенту проще самому решить, сколько памяти он готов отдать. Если места не хватило, метод возвращает ошибку, а уже посчитанная часть решения остается в массивах.

Суффиксы `f` и `l` у методов сделаны по примеру из списка заданий (`pow`, `powf`, `powl`). Тип `ldouble_t` в заголовках Eco.Framework объявлен только для сборки под EcoOS, поэтому он объявлен в моем заголовке под тем же макросом `ECO_LDOUBLE_T_DEFINED`.

3.2. Компоненты

Таблица 2. Компоненты

Компонент	Метод	CID
Eco.Lab1	Рунге-Кутта 4, постоянный шаг	039232DC-C935-4369-9250-13DC847FF6CC
Eco.Lab1DP	Дорман-Принс 4(5), адаптивный шаг	AFC67744-F1C3-43AC-891C-A5DA35D14A4A

Идентификатор интерфейса у обоих компонентов один: IID = 4EF92ACD-8656-4B68-821E-2E480C0651D9. Поэтому клиенту все равно, с каким компонентом он работает: он запрашивает у интерфейсной шины нужный CID и получает указатель на `IEcoAdvancedMath`.

Каркас обоих компонентов создан утилитой eco-wizard (тип COM, язык C). Из шаблона остались фабрика, методы `QueryInterface`, `AddRef`, `Release` и функции создания и удаления экземпляра. Шаблонный интерфейс с методом `MyFunction` заменен на `IEcoAdvancedMath`.

Память берется только через интерфейс `IEcoMemoryAllocator1`. Указатель на него компонент получает у интерфейсной шины в функции инициализации и хранит в поле `m_pIMem`. В методе `Solve` один раз выделяется рабочий массив: $5 \cdot n$ чисел для Рунге-Кутты ($k_1 \ldots k_4$ и промежуточная точка) и $10 \cdot n$ для Дормана-Принса ($k_1 \ldots k_7$, промежуточная точка, новое решение, вектор ошибки). В конце он освобождается.

Отсутствие обращений к libc проверено командой `nm -u` для собранных библиотек: у объектных файлов компонентов нет ни одного внешнего символа, кроме ссылки фабрики на экземпляр компонента.

В языке C89 нет шаблонов, поэтому код методов написан три раза: для `double_t`, `float_t` и `ldouble_t`. Каждая версия считает в своем типе, без преобразования к `double`.

3.3. Коды ошибок

Используются стандартные коды из `ErrEcoCodes.h`.

Таблица 3. Коды возврата

Код	Когда возвращается
ERR_ECO_SUCCESS	решение дошло до конца отрезка
ERR_ECO_POINTER	нулевой указатель: me, f, y0, tOut, yOut
ERR_ECO_INVALIDARG	n = 0, capacity < 2, t0 = t1
ERR_ECO_OUTOFMEMORY	не удалось выделить рабочий массив
ERR_ECO_INDEX_OUT_OF_BOUNDS	не хватило capacity или превышен MaxSteps
ERR_ECO_FAIL	шаг стал меньше точности представления t (только Дорман-Принс)
код из f	правая часть вернула не ноль

4. Клиент

Клиент находится в `Eco.Lab1/UnitTestFiles/SourceFiles/EcoLab1.c`. Это консольное приложение. Оно регистрирует в интерфейсной шине оба компонента и показывает меню из двух режимов.

Первый режим - unit-тесты: 16 групп, всего 50 проверок. Каждая группа печатает таблицу и строки `[OK]` или `[FAIL]`. В клиенте стандартная библиотека разрешена, поэтому точные решения считаются через `exp`, `sin`, `cos` из `math.h`, а время измеряется функцией `clock` из `time.h`.

Второй режим - калькулятор. Пользователь выбирает одну из пяти задач, вводит начальные условия и конец отрезка, выбирает метод и точность. Компонент запрашивается у шины в момент выбора:

```
r = pIBus->pVTbl->QueryComponent(pIBus, method == 1 ? &CID_EcoLab1 : &CID_EcoLab1DP,
                                 0, &IID_IEcoAdvancedMath, (void**) &m);
r = m->pVTbl->Solve(m, funcs[task - 1], &p, n, 0.0, t1, y0, &opt, capacity, g_t, g_y,
&st);
m->pVTbl->Release(m);
```

Пример работы калькулятора: осциллятор на одном периоде, метод Дормана-Принса с RelTol = 10^{-9}.

```
           t                   y1                   y2
    0.000000       1.000000000000       0.000000000000
    1.001311       0.539198395580      -0.842178775556
    3.003788      -0.990519928452      -0.137369104072
    5.199953       0.468475484657       0.883476496201
    6.283185       0.999999999407      -0.000000000054
  шагов 134, отклонено 8, вызовов правой части 853
  точное y1(6.28319) = 1.000000000000, ошибка 5.933e-10
```

В листинге оставлена часть строк, программа печатает около двадцати.

5. Проверка точности

5.1. Порядок метода Рунге-Кутты

Задача $y' = -2y$, $y(0) = 1$ на отрезке $[0, 1]$, точное решение $\exp(-2t)$. Число шагов N каждый раз удваивается.

Таблица 4. Ошибка метода Рунге-Кутты при уменьшении шага

N	h	y(1), компонент	exp(−2), math.h	ошибка	отношение
10	0.1	0.135339548430510	0.135335283236613	4.265e-06	
20	0.05	0.135335528421791	0.135335283236613	2.452e-07	17.40
40	0.025	0.135335297934204	0.135335283236613	1.470e-08	16.68
80	0.0125	0.135335284136256	0.135335283236613	8.996e-10	16.34
160	0.00625	0.135335283292257	0.135335283236613	5.564e-11	16.17
320	0.003125	0.135335283240072	0.135335283236613	3.460e-12	16.08
640	0.0015625	0.135335283236828	0.135335283236613	2.158e-13	16.03

Отношение соседних ошибок стремится к $16 = 2^4$. Это подтверждает 4-й порядок метода. При 640 шагах результат совпадает с `math.h` в 12 знаках после запятой.

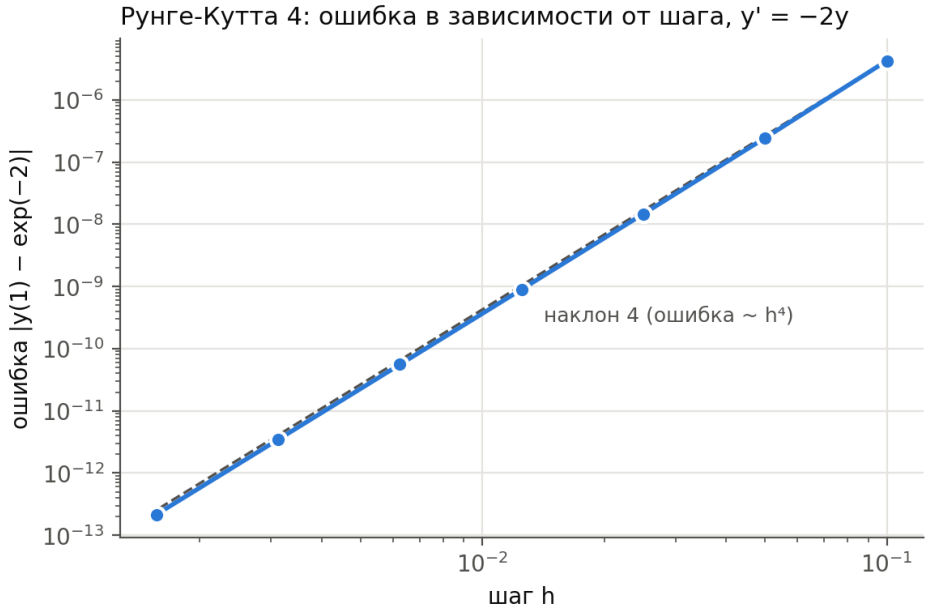


Рис. 1. Ошибка метода Рунге-Кутты в зависимости от шага

5.2. Гармонический осциллятор

Система $y_1' = y_2$, $y_2' = -y_1$ с начальными условиями (1, 0) на отрезке $[0, 10]$, 1000 шагов. Точное решение $y_1 = \cos t$, $y_2 = -\sin t$.

Таблица 5. Осциллятор, метод Рунге-Кутты, 1000 шагов

t	x(t), компонент	cos(t), math.h	ошибка
1	0.540302305937885	0.540302305868140	6.975e-11
2	−0.416146836395020	−0.416146836547142	1.521e-10
4	−0.653643621114055	−0.653643620863612	2.504e-10
6	0.960170286506663	0.960170286650366	1.437e-10
8	−0.145500033148257	−0.145500033808614	6.604e-10
10	−0.839071529523961	−0.839071529076452	4.475e-10

Максимальная ошибка по обеим компонентам на всем отрезке $7.9 \cdot 10^{-10}$, совпадение 9 знаков после запятой. Величина $x^2 + x'^2$ (энергия) должна оставаться равной 1, ее отклонение не превышает $1.4 \cdot 10^{-11}$.

Линейная система $y_1' = -2y_1 + y_2$, $y_2' = y_1 - 2y_2$ с точным решением через $\exp(-t)$ и $\exp(-3t)$ решается на $[0, 2]$ за 200 шагов с ошибкой $6.3 \cdot 10^{-11}$.

5.3. Типы float, double, long double

Та же задача об осцилляторе на [0, 10], значение x(10) сравнивается с cos(10).

Таблица 6. Ошибка для разных типов

Тип	Рунге-Кутта, 1000 шагов	Дорман-Принс: RelTol	шагов	ошибка
float	3.408e-07	1e-06	53	1.328e-06
double	4.475e-10	1e-09	214	8.170e-10
long double	4.475e-10	1e-12	864	7.597e-13

У `float` ошибка ограничена разрядностью самого типа (около 7 значащих цифр). У метода Рунге-Кутты с 1000 шагов `long double` дает тот же результат, что и `double`: ошибку определяет шаг метода, а не точность арифметики. Разница между типами видна у Дормана-Принса при жестком допуске: с `long double` получена ошибка $7.6 \cdot 10^{-13}$.

5.4. Дорман-Принс при разных допусках

Осциллятор на [0, 10]. AbsTol берется в 1000 раз меньше RelTol.

Таблица 7. Метод Дормана-Принса, осциллятор

RelTol	шагов	отклонено	вызовов f	макс. ошибка
1e-03	13	0	79	1.863e-03
1e-04	20	4	145	1.186e-04
1e-05	33	6	235	1.004e-05
1e-06	53	13	397	9.433e-07
1e-07	84	12	577	9.577e-08
1e-08	135	13	889	9.213e-09
1e-09	214	10	1345	8.953e-10
1e-10	342	11	2119	8.765e-11

Полученная ошибка примерно равна заданному допуску во всем диапазоне. При ужесточении допуска в 10 раз число шагов растет примерно в 1.6 раза, это 10^(1/5), как и должно быть у метода 5-го порядка. Число вызовов правой части в каждой строке равно 1 + 6·(шагов + отклонено), это тоже проверяется в тестах.

5.5. Нелинейные системы, сравнение со scipy

У нелинейных задач нет формулы точного решения, поэтому нужен эталон. Вместо MATLAB взят пакет scipy для Python: функция `solve_ivp`. Ее метод RK45 - это тот же метод Дормана-Принса, что в ode45. Эталонные значения посчитаны методом DOP853 (8-й порядок) с допусками 10^{-13}. Для проверки самого эталона те же задачи решены методами LSODA и Radau, результаты совпали до 10^{-11}. Скрипт лежит в `UnitTestFiles/reference.py`.

Таблица 8. Уравнение Ван дер Поля $x'' = (1 - x^2)x' - x$, $x(0) = 2$, $x'(0) = 0$, отрезок [0, 20]

Метод	шагов	вызовов f	x(20)	ошибка
Рунге-Кутта 4	1000	4000	2.008149788763758	7.627e-07
Дорман-Принс, RelTol 1e-8	445	3145	2.008149757319654	5.177e-08
scipy DOP853 (эталон)			2.008149762174939	

Таблица 9. Система Лотки-Вольтерры $x' = 1.5x - xy$, $y' = xy - 3y$, $x(0) = 10$, $y(0) = 5$, отрезок [0, 15]

Метод	шагов	вызовов f	x(15)	ошибка
Рунге-Кутта 4	1000	4000	0.713750617158064	7.609e-07
Дорман-Принс, RelTol 1e-8	529	3259	0.713751378759176	6.614e-10
scipy DOP853 (эталон)			0.713751378097771	

В столбце «ошибка» указано наибольшее отклонение по двум компонентам решения. Оба метода совпадают с эталоном: Рунге-Кутта в 6 знаках, Дорман-Принс в 7-9 знаках, причем за меньшее число вызовов правой части.

На рисунке 2 показано, как метод Дормана-Принса выбирает шаг на уравнении Ван дер Поля. Там, где решение меняется быстро, шаг уменьшается до 0.05, на пологих участках вырастает до 0.28. Метод с постоянным шагом вынужден везде идти самым мелким шагом.

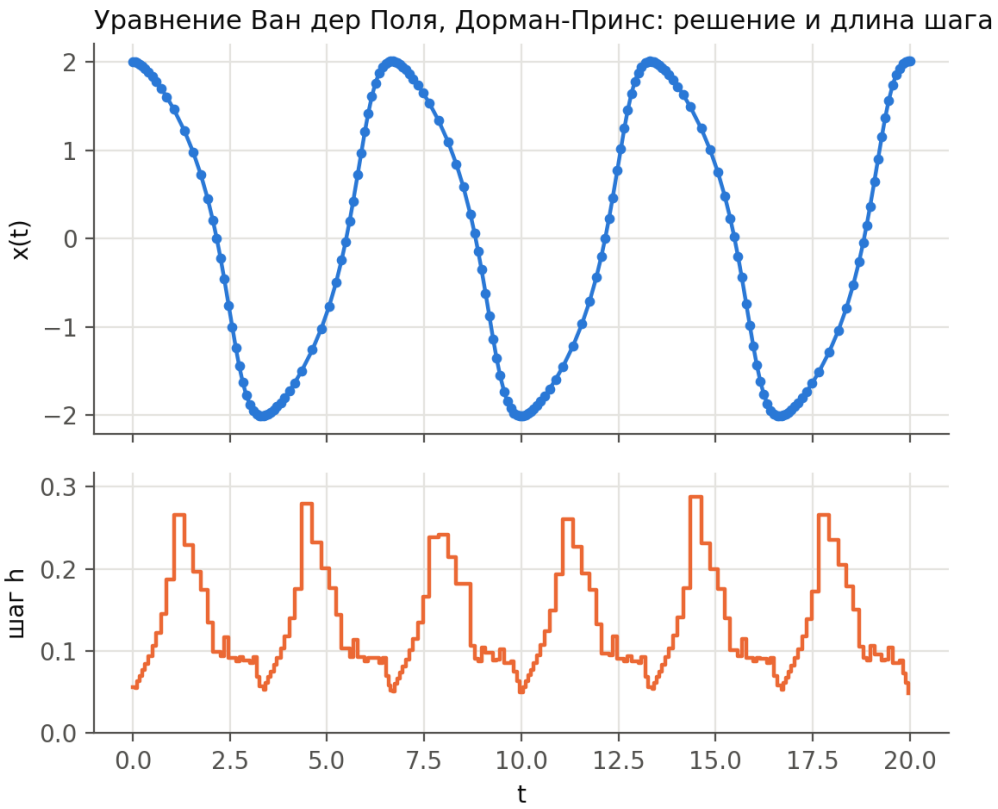


Рис. 2. Решение уравнения Ван дер Поля и длина шага (RelTol = 10^{-6}, 182 шага)

5.6. Обработка ошибок

Для каждого компонента проверено, что неправильные вызовы возвращают ожидаемые коды: нулевые указатели, n = 0, t0 = t1, нехватка места в массивах, превышение `MaxSteps`, ошибка из правой части. Всего 17 проверок. Также проверено интегрирование назад по времени (t1 < t0) и вызов `Step`, когда входной и выходной массив совпадают.

6. Производительность

6.1. Точность и число вызовов правой части

Основная мера затрат для таких методов - число вызовов правой части, потому что в реальных задачах именно она вычисляется дольше всего. На рисунке 3 для осциллятора показано, какую ошибку дает каждый метод при заданном числе вызовов.

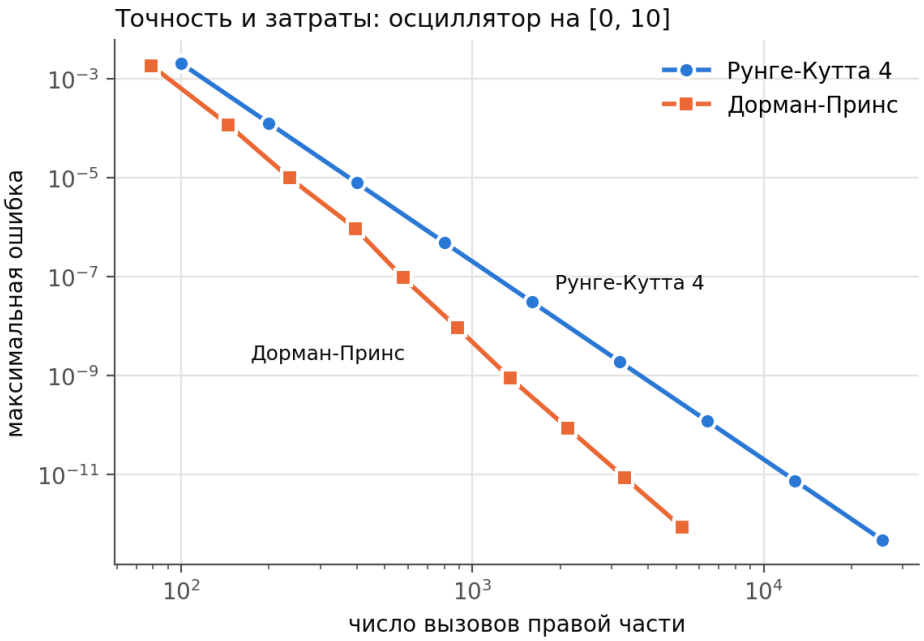


Рис. 3. Ошибка в зависимости от числа вызовов правой части

Линия Дормана-Принса идет круче (5-й порядок против 4-го) и лежит ниже. Для точности 10^{-9} методу Рунге-Кутты нужно 4000 вызовов, Дорману-Принсу 1345, почти в 3 раза меньше. Чем выше требуемая точность, тем больше разрыв.

6.2. Время работы

Время измерялось в клиенте функцией `clock`: каждое решение повторялось 2000 раз, время делилось на число повторов. В таблице 10 приведена медиана по пяти запускам программы. Условия: AMD Ryzen 7 7435HS, Ubuntu 24.04 в WSL2, компилятор gcc 13.2.0 из набора Eco, ключ -O2, статическая сборка. Между запусками время отличается на 5-15 %.

Параметры подобраны так, чтобы точность двух методов была близкой: 1000 шагов у Рунге-Кутты, а у Дормана-Принса RelTol = 10^{-9} для осциллятора и маятника и 10^{-7} для уравнения Ван дер Поля.

Таблица 10. Время одного решения

Задача	Метод	вызовов f	время, мкс	мкс на вызов	ошибка
Осциллятор	Рунге-Кутта 4	4000	46.5	0.012	7.0e-10
	Дорман-Принс	1345	53.5	0.040	8.2e-10
Ван дер Поль	Рунге-Кутта 4	4000	55.8	0.014	7.6e-07
	Дорман-Принс	2185	92.5	0.042	5.4e-07
Маятник	Рунге-Кутта 4	4000	93.1	0.023	4.8e-10
	Дорман-Принс	1165	60.5	0.052	1.4e-09

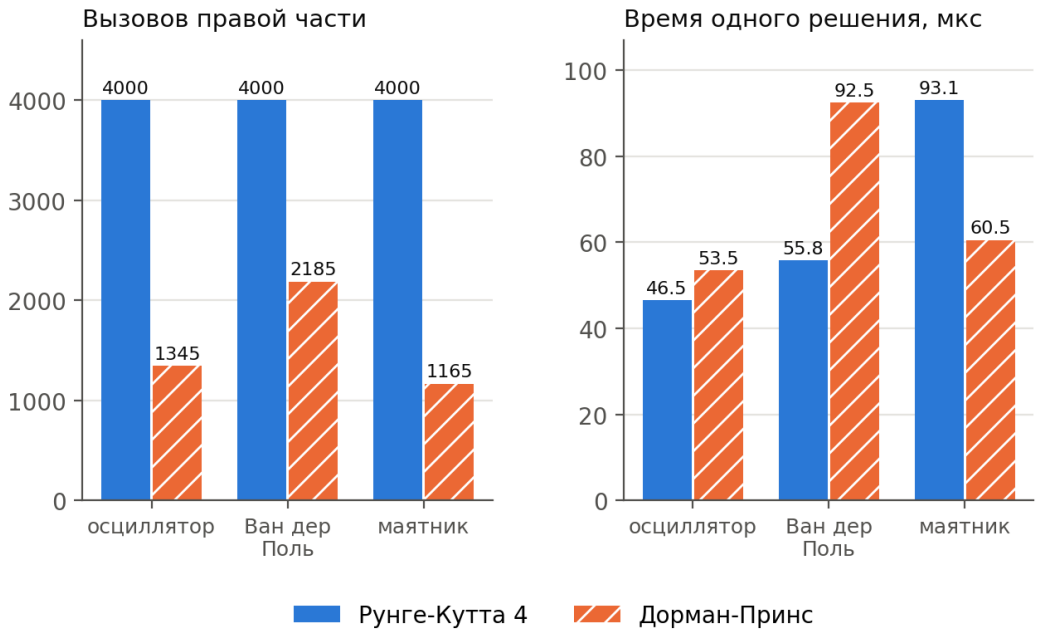


Рис. 4. Число вызовов правой части и время решения

Результат получился не таким, как я ожидал. По числу вызовов Дорман-Принс выигрывает везде, а по времени только на маятнике. Причина в том, что у него больше собственных вычислений на один вызов правой части: 7 стадий с длинными суммами, оценка ошибки, корень 5-й степени. В таблице это видно по столбцу «мкс на вызов»: 0.04 против 0.012.

У осциллятора и уравнения Ван дер Поля правая часть состоит из нескольких умножений и сложений и почти ничего не стоит, поэтому накладные расходы метода определяют все время. У маятника в правой части вызывается `sin`, она заметно дороже, и выигрыш в числе вызовов превращается в выигрыш по времени в 1.5 раза. Вывод: адаптивный метод выгоден тогда, когда правая часть дорогая или когда решение меняется неравномерно. Для простых гладких задач классический метод не хуже.

По ходу замеров в компоненте Дормана-Принса была найдена и исправлена неэффективность. Сначала коэффициенты были записаны в виде `k1[i] * 19372 / 6561`, то есть в цикле выполнялось и умножение, и деление. Компилятор не может сам заменить это одним умножением, потому что результат в плавающей точке немного изменится. После замены на `k1[i] * (19372.0 / 6561)` константа вычисляется при компиляции, и метод стал работать в 1.7 раза быстрее.

6.3. Сравнение со scipy

Те же задачи решены функцией `solve_ivp` с методом RK45 и теми же допусками.

Таблица 11. Компонент Eco.Lab1DP и scipy RK45

Задача	шагов: компонент	шагов: scipy	вызовов f: компонент	вызовов f: scipy	время: компонент	время: scipy
Осциллятор	214	204	1345	1280	53.5 мкс	17 мс
Ван дер Поль	285	272	2185	2102	92.5 мкс	33 мс
Маятник	185	176	1165	1094	60.5 мкс	17 мс

Число шагов и вызовов правой части у компонента и у scipy отличается на 4-7%. Небольшая разница объясняется тем, что scipy нормирует ошибку по среднеквадратичной норме, а в компоненте взят максимум по компонентам, как в ode45. Такое совпадение подтверждает, что метод реализован правильно.

По времени компонент быстрее примерно в 300 раз. Сравнение не совсем честное: в scipy правая часть написана на Python и вызывается через интерпретатор, на это уходит почти все время. Но оно показывает порядок величин.

7. Сборка и запуск

Работа собиралась и проверялась под Linux (Ubuntu 24.04 в WSL2) набором инструментов из материалов курса: gcc 13.2.0, make 4.4.1, Eco.Framework от 18.09.2026. Сначала собирается библиотека второго компонента, затем библиотека первого и клиент:

```
cd Eco.Lab1DP/AssemblyFiles/Linux/gcc_v132
make -f Makefile TARGET=1 DEBUG=0 ARCH=x86_64

cd ../../../../Eco.Lab1/AssemblyFiles/Linux/gcc_v132
make -f Makefile TARGET=1 DEBUG=0 ARCH=x86_64
make -f MakefileExe TARGET=1 DEBUG=0 ARCH=x86_64

../../../BuildFiles/Linux/x86_64/StaticRelease/EcoLab1
```

В `MakefileExe`, который создает eco-wizard, пришлось поправить одну строку. Статические библиотеки Eco ссылаются друг на друга (Eco.System1 использует шину, менеджер памяти и файловую систему), а в командной строке компоновщика они стояли в таком порядке, что символы не находились. Библиотеки объединены в группу ключами `-Wl,--start-group` и `-Wl,--end-group`.

В файлы сборки клиента для Windows (gcc и MSVC), macOS (Xcode) и EcoOS добавлены пути ко второму компоненту и его библиотека. Под Linux сборка проверена полностью. Под Windows компоненты и клиент компилируются компилятором MinGW, но собрать исполняемый файл им нельзя, так как библиотеки Eco.Framework для Windows рассчитаны на компилятор MSVC. Сборку в MSVC и Xcode я не проверял.

8. Выводы

- Спроектирован интерфейс `IEcoAdvancedMath` для решения систем ОДУ и сделаны два компонента АОМК, которые его реализуют: метод Рунге-Кутты 4-го порядка и метод Дормана-Принса 4(5) с адаптивным шагом.
- В компонентах не используется стандартная библиотека, память берется через `IEcoMemoryAllocator1`. Все методы есть для типов `float`, `double` и `long double`.
- Клиент выбирает алгоритм по идентификатору компонента. В нем есть 50 автоматических проверок и интерактивный режим.
- 4-й порядок метода Рунге-Кутты подтвержден: при уменьшении шага вдвое ошибка падает в 16 раз. На задачах с известным решением результат совпадает с `math.h` в 9-12 знаках.
- Метод Дормана-Принса выдерживает заданную точность от 10^{-3} до 10^{-12}, а по числу шагов и вызовов правой части отличается от scipy RK45 на 4-7 %.
- Для точности 10^{-9} Дорману-Принсу нужно примерно в 3 раза меньше вызовов правой части. По времени он выигрывает, когда правая часть дорогая, и проигрывает на самых простых задачах из-за больших собственных вычислений на шаг.

Что можно улучшить: добавить плотный вывод (значения решения в заданных точках, а не только в узлах сетки), как это делает ode45, и убрать дублирование кода для трех типов с помощью макросов.

9. Источники

1. Башев В. Н. Проектирование архитектуры и конструирование программного обеспечения. Лекции 1-3: технология АОМК, компоненты и интерфейсы, интерфейс IEcoUnknown.

2. Хайрер Э., Нёрсетт С., Ваннер Г. Решение обыкновенных дифференциальных уравнений. Нежесткие задачи. М.: Мир, 1990.

3. Dormand J. R., Prince P. J. A family of embedded Runge-Kutta formulae // Journal of Computational and Applied Mathematics. 1980. Vol. 6, no. 1. P. 19-26.

4. Runge-Kutta methods. Wikipedia. https://en.wikipedia.org/wiki/Runge-Kutta_methods

5. Dormand-Prince method. Wikipedia. https://en.wikipedia.org/wiki/Dormand-Prince_method

6. MATLAB documentation: ode45. https://www.mathworks.com/help/matlab/ref/ode45.html

7. SciPy documentation: scipy.integrate.solve_ivp.
https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html